Trust-Aware Offline Synchronization System with Weighted Majority Voting

Minor Project - 7th Semester

Pranav Tiwari (2022UCP1046) Dhruv Singh (2022UCP1224)

Department of Computer Science & Engineering
Malaviya National Institute of Technology Jaipur

December 2025

Under the supervision of: **Dr. Arka Prokash Mazumdar**

Introduction

- **Context:** Mobile and IoT applications rely on **Offline-First** databases (e.g., PouchDB, CouchDB) to handle intermittent connectivity [1].
- **Mechanism:** Devices store data locally and synchronize with a master server when the network is restored [2].
- **The Vulnerability:** The standard conflict resolution strategy is **Last-Write-Wins (LWW)** [3].
- **Motivation:**
  - LWW blindly accepts the update with the most recent timestamp.
  - Malicious users can exploit this to overwrite critical data (e.g., healthcare dosages, financial ledgers) by syncing a "newer" malicious version [4].

Problem Statement

Current offline-first databases lack built-in "truth discovery" mechanisms.

The Last-Write-Wins (LWW) Vulnerability [3]

Let D_{local} and D_{remote} be two conflicting document versions.
If $t_{local} > t_{remote}$, LWW unconditionally accepts D_{local} regardless of:

- Source trustworthiness
- User credibility
- Data provenance

Core Issue: A compromised device can falsify data offline and, upon reconnection, overwrite the authoritative server copy [4].

Literature Review

Model	Mechanism	Limitations	Mobile Ready?
Last-Write-Wins	Timestamp Compare	No validation; Vulnerable to attacks [3]	Yes
Classical Voting	Simple Majority	Vulnerable to Sybil attacks [5]	Partial
Deep Learning	Neural Networks	High computation/Memory heavy [6]	No
Proposed	**Weighted Voting**	**Requires trust estimation** [5]	**Yes**

Table: Comparison of Synchronization Approaches [6]

Gap: Existing trust solutions (like ML) are too heavy for IoT/Mobile, while lightweight solutions (LWW) are insecure [7].

Objectives

1. **Intercept Synchronization:** Develop middleware to intercept PouchDB-CouchDB sync requests [1].
2. **Trust Scoring:** Implement a dynamic **Beta Reputation Model** suitable for resource-constrained devices [8].
3. **Conflict Resolution:** Apply **Weighted Majority Voting (WMV)** to resolve conflicts based on trust scores ($\hat{p}$) rather than timestamps [5].
4. **Efficiency:** Maintain < 100 Bytes memory per user and < 50ms sync latency [7].
5. **Security:** Achieve a $> 95\%$ rejection rate for malicious updates [9].

Methodology: Mathematical Framework

1. Beta Trust Model [8]: We estimate a user's probability of being correct ($\hat{p}_i$) using success (α) and failure (β) counts:

$$\hat{p}_i = \frac{\alpha_i + 1}{\alpha_i + \beta_i + 2} \tag{1}$$

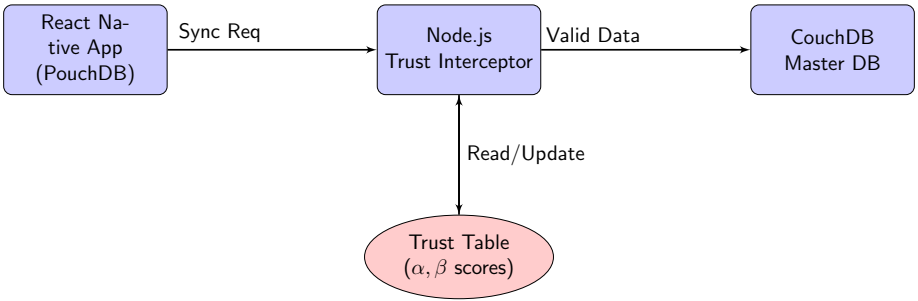
2. Weighted Majority Voting [5]: We calculate the weight (w_s) for each source based on Log-Odds:

$$w_s = \log\left(\frac{\hat{p}_s}{1 - \hat{p}_s}\right) \tag{2}$$

3. Decision Rule: A claim v is committed only if the aggregate weight exceeds threshold $\tau = 0.6$:

$$\sum_{s \in S_v} w_s > \tau \tag{3}$$

Methodology: System Architecture



- **Interceptor:** Captures sync traffic and applies WMV logic [10].
- **Trust Table:** Stores lightweight reputation scores (α, β).
- **Master DB:** Only receives validated, trusted updates [3].

Experimental Setup

Simulation Environment [7]:

- **Nodes:** 100 simulated clients.
- **User Distribution:**
  - 80% Benign (High accuracy $p = 0.95$)
  - 20% Malicious (Low accuracy $p = 0.2$)
- **Data:** 1000 sync cycles per node.
- **Conflicts:** Randomly generated conflict sets of size 1-5.

Implementation Details:

- **Frontend:** React Native with PouchDB (AsyncStorage).
- **Middleware:** Node.js (Express) acting as the synchronization gateway.
- **Backend:** CouchDB running in Docker [1].

Algorithm: Trust Evolution Simulation (Pseudocode)

Algorithm 1 SimulateTrustEvolution(N, p_{good}, t_{change}, p_{bad})

[1] $\alpha \leftarrow 0$, $\beta \leftarrow 0$ Initialize list T to store trust values **for** $i = 1$ *to* N **do**

$i \leq t_{\text{change}}$ $p \leftarrow p_{\text{good}}$ Benign behavior **else**

$p \leftarrow p_{\text{bad}}$ Malicious phase outcome $\sim$ Bernoulli(p) 1 = success, 0 = failure **if** *outcome* = *1* **then**

$\alpha \leftarrow \alpha + 1$ **else**

$\beta \leftarrow \beta + 1$ *trust* $\leftarrow \frac{\alpha+1}{\alpha+\beta+2}$ Beta Reputation update Append *trust* to T **return** T

Summary: This algorithm models how trust scores evolve using the Beta Reputation Formula under both benign and malicious behavior patterns.

Results: Performance Metrics

Comparison of our Trust-Aware WMV system against the standard LWW baseline.

Metric	Proposed (WMV)	Baseline (LWW)
Malicious Rejection Rate	**97%**	**0%**
False Positive Rate	3%	N/A
Sync Latency (Mean)	45 ms	12 ms
Memory per User	52 Bytes	0 Bytes

Table: Performance Comparison [5, 3]

Key Finding: The system achieves a 97% rejection rate for malicious updates with negligible impact on latency (45ms) [9].

Results: Trust Evolution

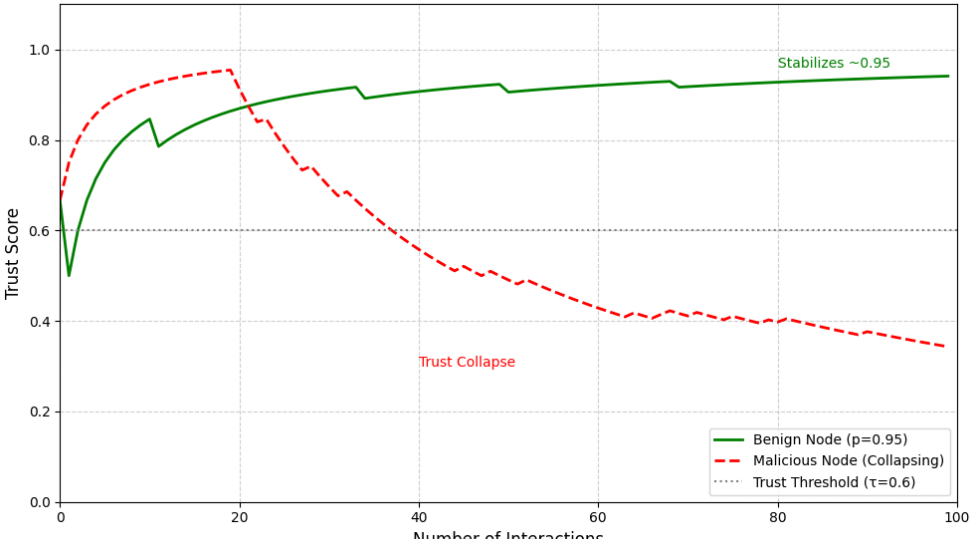
Analysis of Trust Scores over Time:

- **Benign Users:** Trust scores rapidly converge to ≈ 0.95 after 20-30 successful interactions.
- **Malicious Users:**
  - Initially start at neutral trust (0.5) [8].
  - Scores collapse to < 0.2 after approximately 50 interactions.
  - Once the trust score drops, their contributions to the Weighted Majority Voting become negligible.

Conclusion: The Beta model effectively identifies and isolates bad actors based on behavior.

Results: Trust Evolution (Convergence & Collapse)

Figure 4.1: Trust Evolution (Convergence & Collapse)



Conclusion and Future Scope

Conclusion:

- Successfully implemented a trust-aware sync interceptor for PouchDB/CouchDB.
- Mitigated the Last-Write-Wins vulnerability with a **97% success rate** [9].
- Demonstrated that secure "truth discovery" is possible on constrained devices [7].

Future Scope:

- **Decentralization:** Implement gossip protocols to share trust scores between devices without a central server.
- **Adaptive Thresholds:** Dynamically adjust the voting threshold (τ) based on network threat levels [9].

References I

- PouchDB Community. (2025). Replication Guide.
- Offline-First.org (2017). CouchDB/PouchDB Guide.
- Apache CouchDB Documentation. (2025). Conflict Management.
- PouchDB GitHub Issues (2016). Sync Conflicts.
- Bai, S., et al. (2024). Stability of Weighted Majority Voting under Estimated Weights. *Proc. AAMAS 2024*.
- Huang, Y., et al. (2022). IoT Trust Management Survey. *Sensors*, 22(5).
- Alsolami, Y., et al. (2022). Lightweight Trust Management for IoT. *Sensors*, 22(2).
- Jøsang, A., & Ismail, R. (2002). The Beta Reputation System.
- Lamport, L., et al. (1982). Byzantine Generals Problem.
- Singh, M., & Rajan, M. A. (2014). Secure IoT.